FRANCELLA ROJAS C

ENTORNOS DE DESARROLLO

DAW 1

REFACTORIZACIÓN

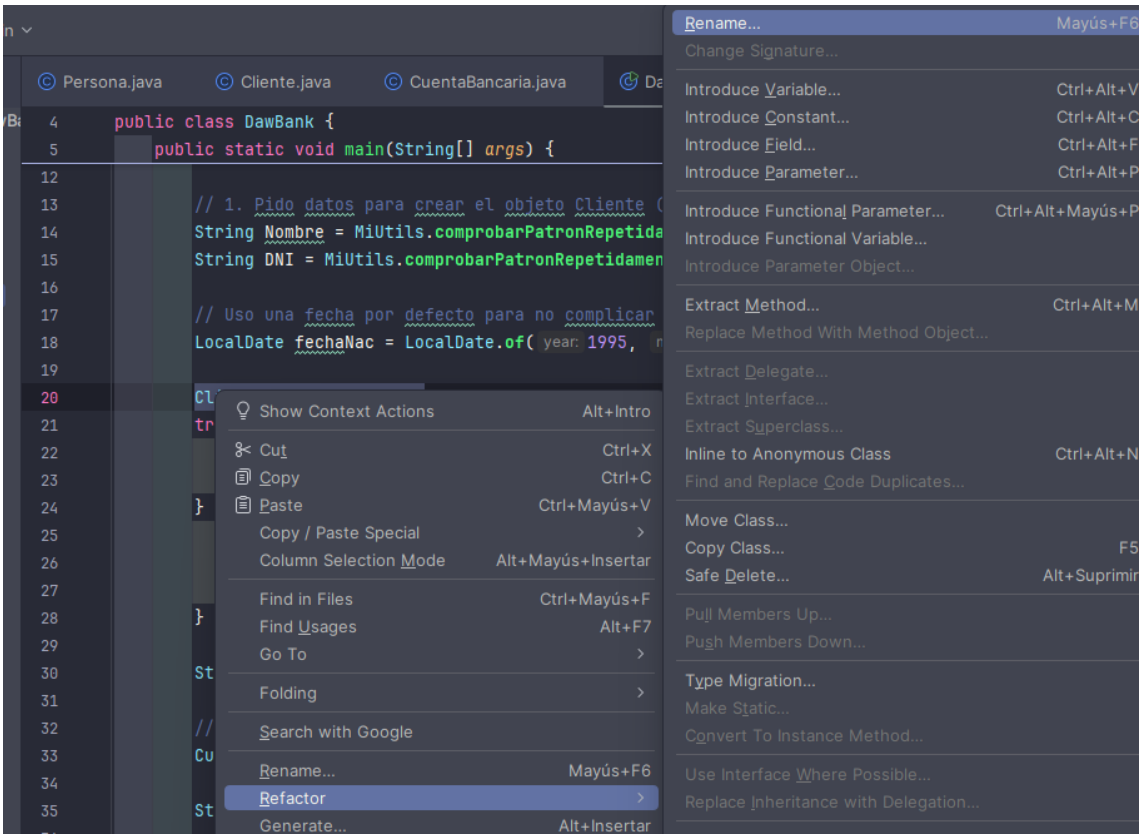
CASO 1:

Rename

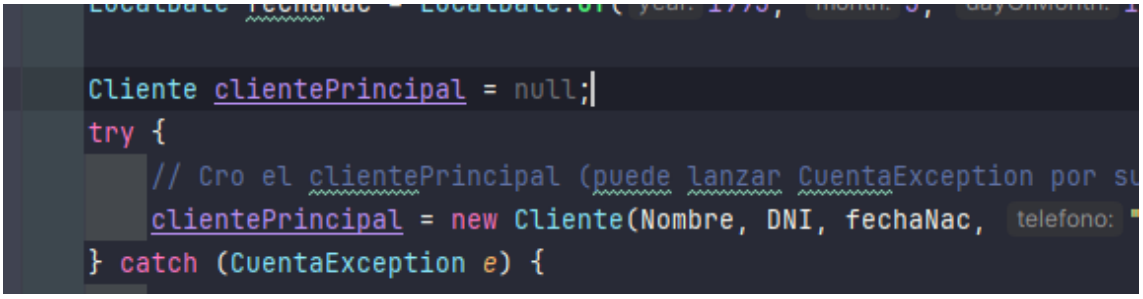
Esta opción permite cambiar el nombre a cualquier elemento de nuestro código, tanto campos, variables, métodos, clases, paquetes, ficheros, etc.

El sistema se encarga de cambiar el nombre en todos los objetos del proyecto, así como las referencias. Por ejemplo, si cambiamos el nombre de una clase cambiará el nombre del constructor. Si cambiamos el nombre del paquete cambiarán las referencias de este en el resto de clases.

En este caso realizare un refactor de RENAME para una variable.



Y aquí veo como pasó de llamarse “cliente” a “clientePrincipal” en todas las veces que aparece en el código.

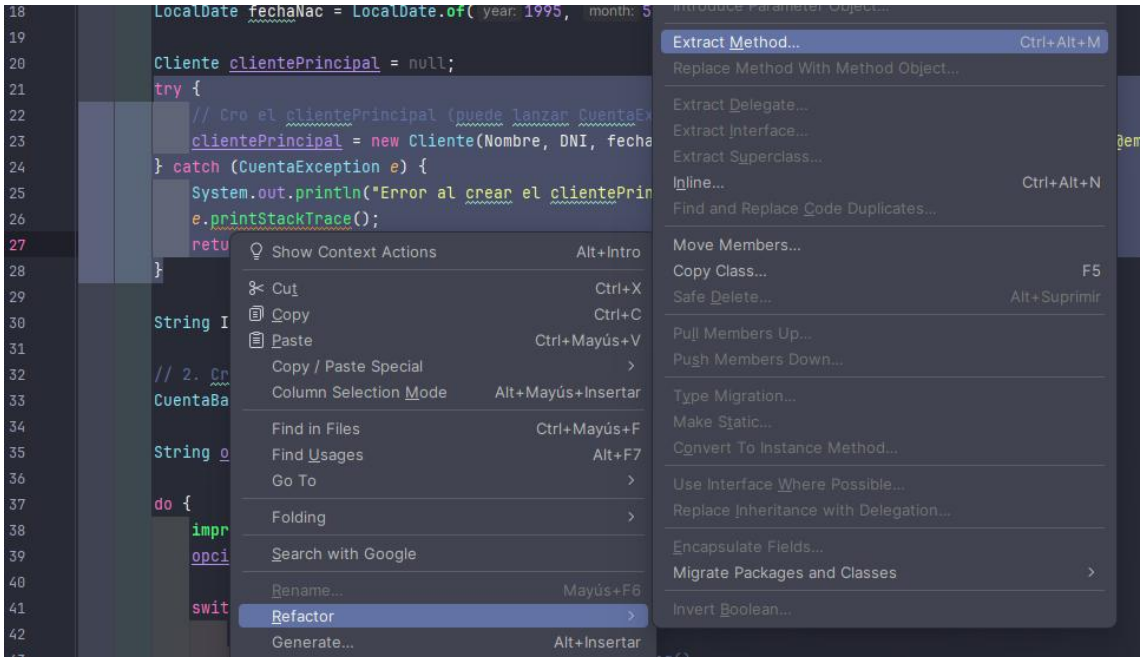


CASO 2:

En este caso voy a tomar un try catch, y voy a hacerlo un método.

```
21        try {
22            // Cro el clientePrincipal (puede lanzar CuentaException por sus setters)
23            clientePrincipal = new Cliente(Nombre, DNI, fechaNac,  telefono: "600123123",  email: "clientePrincipal@email.com",  direccion: "Calle Mayor 1");
24        } catch (CuentaException e) {
25            System.out.println("Error al crear el clientePrincipal: " + e.getMessage());
26            e.printStackTrace();
27            return; // Si no hay clientePrincipal, no puedo seguir
28        }
```

Con Refactor, Extraer Método.



Aquí vemos como se creó, haciendo que yo pueda reutilizar el método cuantas veces quiera y limpiando el código principal.

```
21        clientePrincipal = tryMetodoPruebas(Nombre, DNI, fechaNac);
```

```
77
78 @   private static Cliente tryMetodoPruebas(String Nombre, String DNI, LocalDate fechaNac) {  1 usage
79         Cliente clientePrincipal;
80         try {
81             // Cro el clientePrincipal (puede lanzar CuentaException por sus setters)
82             clientePrincipal = new Cliente(Nombre, DNI, fechaNac,  telefono: "600123123",  email: "clientePrincipal@email.com",  direccion: "Calle Mayor 1");
83         } catch (CuentaException e) {
84             System.out.println("Error al crear el clientePrincipal: " + e.getMessage());
85             e.printStackTrace();                    Typo: In word 'cliente'
86             return null;
87         }
88         return clientePrincipal;
89     }
```

CASO 3:

Extract interface

La refactorización *extract interface* permite seleccionar métodos públicos no estáticos y moverlos a una interfaz independiente. Esto puede hacer que el código sea más legible y fácil de mantener.

En este caso quise probar este con un método público no estático a ver que hacia:

```
1   import java.time.LocalDate;
2
3   public class Cliente extends Persona {  6 usages
4       private String telefono;  3 usages
5       private String email;  3 usages
6       private String direccion;  3 usages
7
8
9       //CONSTRUCTOR
```

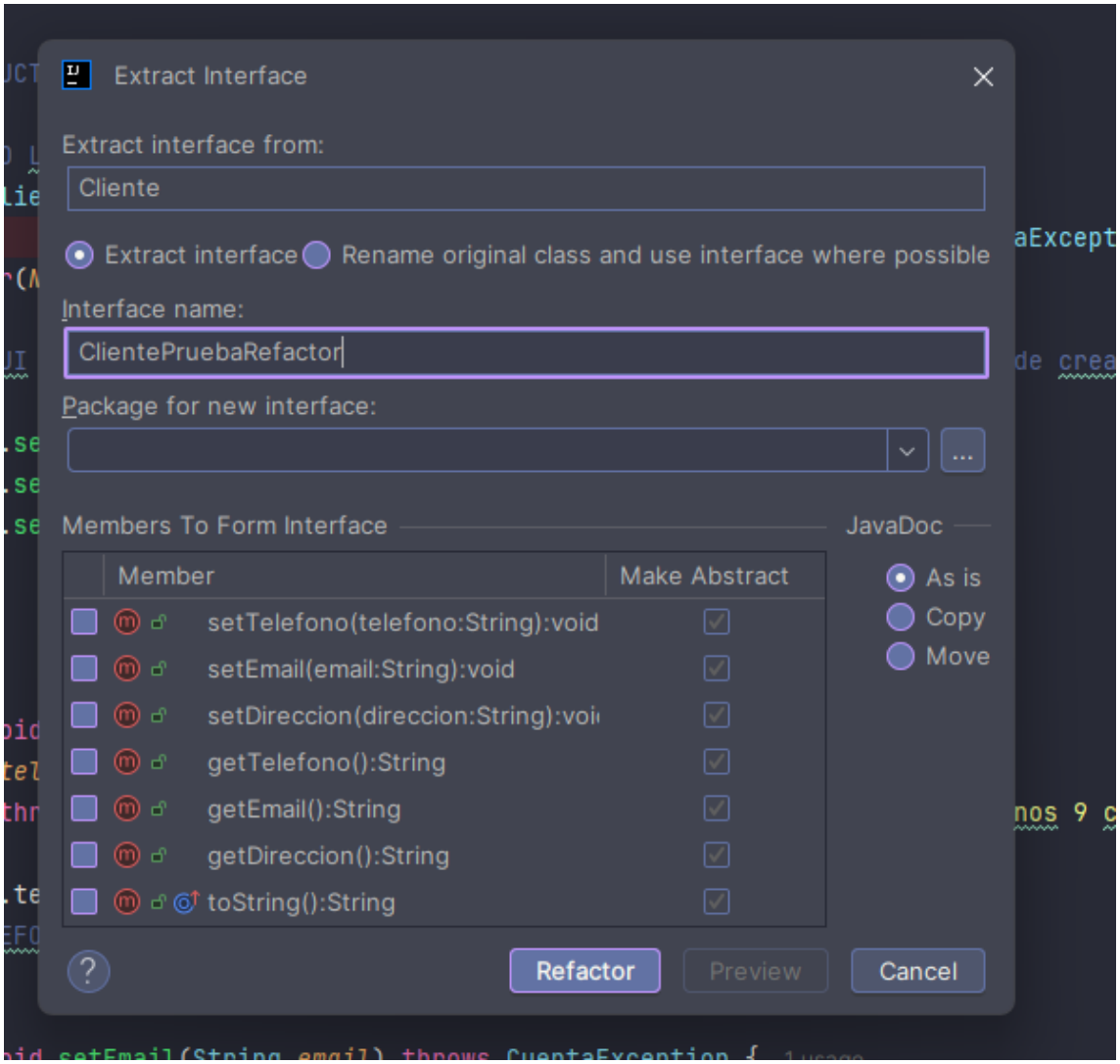
Quiero hacerlo para la clase Cliente:

```
11      //PRIMERO LLA
12      public Client
13
14          super(Nom
15
16          //AQUI CR
17
18          this.setT
19          this.setE
20          this.setD
21      }
22
23  //SETTERS
24
25 @    public void setTelefono(String telefono) throws CuentaExce
26          if (telefono.length() < 9 || !telefono.matches( regex: "
27              throw new CuentaException("El telefono no es valid
```

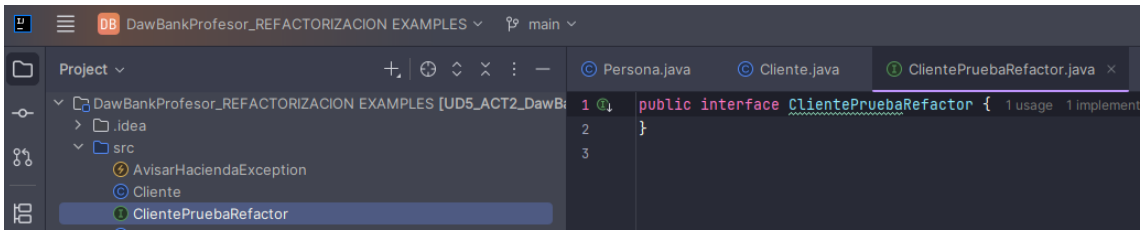
```
Folding
Analyze
Rename...                    Mayús+F6
Refactor
Generate...                  Alt+Insertar
Open In
Local History
Compare with Clipboard
Diagrams
Create Gist...
```

```
Rename...                    Mayús+F6
Change Signature...
Introduce Variable...        Ctrl+Alt+V
Introduce Constant...        Ctrl+Alt+C
Introduce Field...           Ctrl+Alt+F
Introduce Parameter...       Ctrl+Alt+P
Introduce Functional Parameter...   Ctrl+Alt+Mayús+P
Introduce Functional Variable...
Introduce Parameter Object...
Extract Method...            Ctrl+Alt+M
Replace Method With Method Object...
Extract Delegate...
Extract Interface...
Extract Superclass...
```

Y me da la opción de elegir nombre y que cosas incluir o no:



Y aquí puedo ver que me ha creado la interfaz



Otra cosa que he notado, es que las opciones en el menú refactor, cambian según el código que se tenga seleccionado, imagino que el IDE es lo suficientemente listo como para saber que puede o no hacer con cierta selección.